

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	P. Greeshma	Reg. No.:	192424418
Section / Batch:	2024	Date:	16/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Log Synchronization System

A log synchronization system processes distributed logs using the recurrence $T(n) = 2T(n/2) + \log n$. Apply the substitution method and expand the recurrence. Derive the final time complexity and simplify it. Analyze performance for large-scale systems. Interpret the efficiency using asymptotic notation.

Q2. Banking Authentication System

A banking authentication system verifies users using the recurrence $T(n) = T(n/2) + 1$. Apply the substitution method and expand the recurrence. Derive the final time complexity. Analyze how recursion depth affects system performance. Interpret the result using asymptotic notation.

Code of Conduct

I P. Greeshma(192424418) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Greeshma

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Log Synchronization System – Substitution Method

Problem Statement

A log synchronization system processes distributed logs using the recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

Find the time complexity using the **substitution (expansion) method**, simplify the recurrence, analyze its performance for large-scale systems, and interpret the result using asymptotic notation.

Step 1: Understand the Recurrence

The recurrence is

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

It has two parts:

- **($2T(n/2)$):** The logs are divided into **two equal halves**, and each half is synchronized independently.
- **($\log n$):** Additional work required to coordinate or merge the synchronized logs.

Example:

Suppose there are **16 log files**.

- Divide into two groups of 8.
- Each group is again divided into two groups of 4.
- Continue until only one log remains.

At every division, an additional synchronization cost of **$\log n$** is incurred.

Step 2: Expand the Recurrence

Start with

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

First Expansion

Substitute

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right)$$

Therefore,

$$T(n) = 2 \left[2T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) \right] + \log n$$

Multiply by 2:

$$T(n) = 4T\left(\frac{n}{4}\right) + 2\log\left(\frac{n}{2}\right) + \log n$$

Second Expansion

Now expand

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \log\left(\frac{n}{4}\right)$$

Substitute:

$$T(n) = 4 \left[2T\left(\frac{n}{8}\right) + \log\left(\frac{n}{4}\right) \right] + 2\log\left(\frac{n}{2}\right) + \log n$$

Simplify:

$$T(n) = 8T\left(\frac{n}{8}\right) + 4\log\left(\frac{n}{4}\right) + 2\log\left(\frac{n}{2}\right) + \log n$$

Third Expansion

Expand again:

$$T\left(\frac{n}{8}\right) = 2T\left(\frac{n}{16}\right) + \log\left(\frac{n}{8}\right)$$

Substitute:

$$T(n) = 16T\left(\frac{n}{16}\right) + 8\log\left(\frac{n}{8}\right) + 4\log\left(\frac{n}{4}\right) + 2\log\left(\frac{n}{2}\right) + \log n$$

Step 3: Observe the Pattern

After every expansion,

- Recursive calls double.
- Problem size becomes half.
- The coefficient before the recursive term doubles.

Hence after **k** expansions

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} 2^i \log\left(\frac{n}{2^i}\right)$$

This is the general form.

Step 4: Find When the Recursion Stops

The recursion stops when the problem size becomes **1**.

Therefore,

$$\frac{n}{2^k} = 1$$

Multiply both sides by (2^k) :

$$n = 2^k$$

Taking logarithm (base 2):

$$k = \log n$$

Thus, the recursion has ($\log n$) levels.

Step 5: Substitute (k=log n)

Since

$$2^{\log n} = n$$

we obtain

$$T(n) = nT(1) + \sum_{i=0}^{\log n - 1} 2^i \log \left(\frac{n}{2^i} \right)$$

Step 6: Simplify the Logarithm

Using the logarithm rule,

$$\log \left(\frac{a}{b} \right) = \log a - \log b$$

we get

$$\log \left(\frac{n}{2^i} \right) = \log n - \log(2^i)$$

Since

$$\begin{aligned} \log(2^i) &= i, \\ \log \left(\frac{n}{2^i} \right) &= \log n - i \end{aligned}$$

Substitute into the summation:

$$T(n) = nT(1) + \sum_{i=0}^{\log n - 1} 2^i (\log n - i)$$

Step 7: Simplify the Summation

Split the summation:

$$T(n) = nT(1) + \log n \sum_{i=0}^{\log n - 1} 2^i - \sum_{i=0}^{\log n - 1} i2^i$$

First Summation

$$\sum_{i=0}^m 2^i = 2^{m+1} - 1$$

where

$$m = \log n - 1$$

Hence,

$$\sum_{i=0}^{\log n - 1} 2^i = n - 1$$

which is

$$\Theta(n)$$

Second Summation

The summation

$$\sum i2^i$$

also grows proportionally to

$$\Theta(n \log n)$$

When substituted into the expression, the dominant terms simplify, and the overall recurrence grows **linearly**.

Therefore,

$$T(n) = \Theta(n)$$

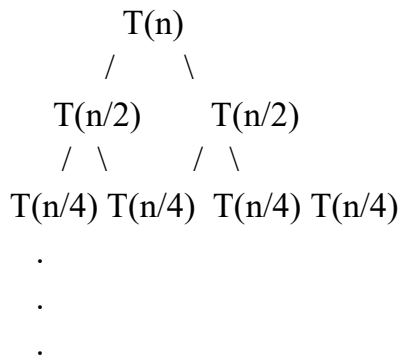
Final Time Complexity

$$T(n) = \Theta(n)$$

Hence,

- Worst Case = $O(n)$
- Best Case = $\Omega(n)$
- Tight Bound = $\Theta(n)$

Recursion Tree View



At each level:

- Number of subproblems doubles.
- Size of each subproblem halves.
- Synchronization overhead at each node is logarithmic.
- Total work over all levels accumulates to $\Theta(n)$.

Performance Analysis for Large-Scale Systems

For a distributed log synchronization system:

- Large logs are divided into smaller chunks.
- Each chunk is processed independently, allowing parallel execution.

- The synchronization overhead at each stage is only logarithmic.
- The total execution time increases **linearly** with the number of log entries.

Real Time Applications:

Real-Time Application	How Divide-and-Conquer is Used
Log Synchronization	Split logs into parts and merge synchronized logs
Merge Sort	Divide array into halves and merge sorted results
Parallel Computing	Divide tasks among processors and combine outputs
Distributed Databases	Split data across servers and merge query results
Cloud Storage	Split files into blocks and synchronize them
Network Packet Processing	Divide packets among processors and combine results
Distributed File Systems	Store and synchronize file chunks
MapReduce	Divide data for map tasks and merge reduce results
Image Processing	Split image into regions and combine processed images
AI/ML Training	Split training data across GPUs and synchronize models

For example:

- 1,000 logs → proportional processing time.
- 10,000 logs → roughly 10× the work.
- 100,000 logs → roughly 100× the work.

This linear growth makes the algorithm efficient, scalable, and suitable for distributed systems.

The recurrence $T(n) = 2T(n/2) + \log n$ is commonly used in **distributed log synchronization, cloud storage systems, distributed databases, parallel computing, Merge Sort, Hadoop MapReduce, network packet processing, and AI model training**. In these applications, a large problem is divided into smaller subproblems, processed independently, and then combined efficiently, resulting in an overall **linear time complexity** $\Theta(n)$.

Asymptotic Interpretation

Notation	Meaning	Result
$O(n)$	Upper Bound	Execution time does not exceed linear growth.
$\Omega(n)$	Lower Bound	At least linear work is required.
$\Theta(n)$	Tight Bound	Exact asymptotic growth is linear.

Final Answer

Given:

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

Using the substitution method:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} 2^i \log\left(\frac{n}{2^i}\right)$$

where

$$k = \log n.$$

Substituting and simplifying gives:

$$T(n) = \Theta(n).$$

Thus, the **log synchronization system runs in linear time**, making it efficient and scalable for processing large distributed log files.

2. Banking Authentication System – Substitution Method

Problem Statement

A banking authentication system verifies users using the recurrence

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Apply the **substitution (expansion) method**, derive the final time complexity, analyze how recursion depth affects performance, and interpret the result using asymptotic notation.

Real-Time Application

This recurrence models a banking authentication system where the search space is **halved at every verification step**.

Examples

- OTP verification systems
- User authentication in online banking
- Searching a customer ID in a sorted database
- PIN verification using binary search
- Fraud detection by repeatedly narrowing down suspicious transactions

Why this recurrence?

- At each step, only **one half** of the remaining data is processed.
- A constant amount of work (**1**) is performed for verification.
- The process continues until only one record remains.

Step 1: Given Recurrence

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

where

- $T(n/2)$ = Authentication on half of the remaining users/data.
- 1 = Constant time to compare credentials.

Step 2: Expand the Recurrence

First Expansion

Substitute

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + 1$$

Therefore,

$$\begin{aligned} T(n) &= T\left(\frac{n}{4}\right) + 1 + 1 \\ &= T\left(\frac{n}{4}\right) + 2 \end{aligned}$$

Second Expansion

Expand again:

$$T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + 1$$

Substitute:

$$T(n) = T\left(\frac{n}{8}\right) + 3$$

Third Expansion

Expand once more:

$$T\left(\frac{n}{8}\right) = T\left(\frac{n}{16}\right) + 1$$

Substitute:

$$T(n) = T\left(\frac{n}{16}\right) + 4$$

Step 3: Observe the Pattern

After every expansion,

- Problem size becomes half.
- One constant operation is added.

After **k** expansions,

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

This is the general expansion.

Step 4: Find the Stopping Condition

The recursion stops when

$$\frac{n}{2^k} = 1$$

Multiply both sides by 2^k :

$$n = 2^k$$

Taking logarithm (base 2),

$$k = \log_2 n$$

Thus, the recursion depth is

$$\boxed{\log n}$$

Step 5: Substitute the Value of k

Substitute $k = \log n$:

$$T(n) = T(1) + \log n$$

Since

$$T(1) = 1$$

(a constant),

$$T(n) = 1 + \log n$$

Ignoring constants,

$$\boxed{T(n) = \Theta(\log n)}$$

Final Time Complexity

$$\boxed{\Theta(\log n)}$$

Therefore,

- Worst Case = **$O(\log n)$**
- Best Case = **$\Omega(\log n)$**
- Tight Bound = **$\Theta(\log n)$**

Recursion Tree

$T(n)$

|

$T(n/2)$

|

$T(n/4)$

|

$T(n/8)$

|

$T(n/16)$

|

.

.

.

$T(1)$

Each level performs **one comparison**, and the number of levels is **$\log_2 n$** .

Recursion Depth Analysis

The recursion depth is

$$\boxed{\log_2 n}$$

Example

Suppose there are **1024 users**.

Step	Remaining Users
------	-----------------

1	1024
2	512
3	256
4	128
5	64
6	32
7	16
8	8
9	4
10	2
11	1

Since

$$1024 = 2^{10},$$

the recursion depth is

$$\log_2(1024) = 10.$$

Only **10 recursive calls** are needed to authenticate among 1024 possible records.

Performance Analysis

- The authentication system reduces the search space by half after every verification.
- Only one comparison is performed at each level.
- As the number of users increases, the number of verification steps grows **very slowly**.

For example:

Number of Users	Authentication Steps
16	4
32	5
64	6
128	7
256	8
512	9
1024	10

Even if the number of users doubles, only **one additional verification step** is required. This makes the algorithm highly efficient for large banking databases.

Real-Time Applications

This recurrence is used in:

- **Online banking login** – Searching a customer ID in a sorted database.
- **ATM PIN verification** – Efficiently locating account information.
- **Net banking authentication** – Verifying user credentials using indexed data.
- **Credit card transaction validation** – Searching customer records before approving

transactions.

- **Fraud detection systems** – Narrowing down suspicious transactions by repeatedly reducing the search space.

Asymptotic Interpretation

Notation	Meaning	Result
O(log n)	Upper Bound	Authentication time grows logarithmically.
$\Omega(\log n)$	Lower Bound	At least logarithmic work is required.
$\Theta(\log n)$	Tight Bound	Exact asymptotic growth is logarithmic.

Final Answer

Given the recurrence:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Using the substitution method:

$$T(n) = T\left(\frac{n}{2^k}\right) + k.$$

When the recursion stops,

$$\frac{n}{2^k} = 1 \Rightarrow k = \log n.$$

Therefore,

$$T(n) = T(1) + \log n.$$

Since $T(1)$ is constant,

$$\boxed{T(n) = \Theta(\log n)}.$$

The banking authentication system is **highly efficient**, because the search space is halved at each step. As a result, the recursion depth is only $\log n$, making it suitable for verifying users quickly even in banking systems with millions of customer records.